

Cryptography and Security

Blockchain Hackathon — Getting started

February 22, 2026

Contents

1	Background	1
2	Set up your wallet and get testnet funds	2
3	Send a transaction	3
4	Deploy a smart contract	3
5	Interact with your smart contract	5

In this hackathon session, you will get hands-on experience with the Ethereum blockchain. You will set up a wallet, obtain test funds, send a transaction, write and deploy a smart contract, and interact with it, all on the Sepolia testnet. No prior blockchain knowledge is assumed.

1 Background

Blockchain. A blockchain is a distributed, append-only ledger maintained by a peer-to-peer network. There is no central authority: nodes reach consensus through a protocol.

- **Bitcoin** is the first and largest blockchain, created in 2009 by the pseudonymous Satoshi Nakamoto. It was designed primarily as a decentralised peer-to-peer electronic cash system and remains the reference implementation of the technology.
- **Ethereum** is a programmable blockchain that goes beyond simple payments: at its core runs the Ethereum virtual machine (EVM), a shared computation engine replicated across all nodes in the network. Every node runs the same code and reaches the same result, which means programs (called *smart contracts*) execute exactly as written, without any central authority controlling them. The native currency of Ethereum is Ether (ETH).

Block explorers. A block explorer is a website that lets anyone inspect the contents of a blockchain in a human-readable way: transactions, blocks, addresses, smart contracts, and more. In public blockchain (like Bitcoin and Ethereum), everything is visible to everyone, no login is required.

- **Etherscan** (<https://etherscan.io>) is the most widely used explorer for Ethereum. Any time you send a transaction or deploy a contract in Ethereum (or in Sepolia), you will use Etherscan to verify that it was successfully included in the blockchain.

Wallets. A wallet is a piece of software (or hardware) that allows users to interact with the blockchain. It manages your private keys, builds and signs transactions, and shows your balances.

- **MetaMask** is a browser-extension wallet that supports Ethereum and all EVM-compatible networks. It is a hierarchical deterministic wallet: a single secret *mnemonic phrase* (12 or 24 words, defined by the BIP-39 standard) is used to derive all key pairs.

Testnets. Testnets are public blockchains that mimic the main network but use valueless tokens.

- **Sepolia** is the standard Ethereum testnet. You can get free SepoliaETH from a *faucet*.

Solidity. Solidity is the main programming language for writing smart contracts on Ethereum. It is a statically-typed, compiled language with a syntax inspired by JavaScript and C++, designed specifically to run on the EVM.

- **Remix** (<https://remix.ethereum.org>) is a browser-based integrated development environment for writing, compiling, deploying, and interacting with Solidity smart contracts. It connects to MetaMask to sign and broadcast transactions to the network.

Gas. Every operation on Ethereum consumes *gas*, a unit measuring computational effort. Users pay gas fees in ETH, which are calculated as

$$\text{gas fee} = \text{gas units} \times \text{gas price}.$$

Gas prevents abuse (due to the *halting problem*) by requiring payment proportional to computation. Gas prices fluctuate with network congestion.

2 Set up your wallet and get testnet funds

In this first part you will install MetaMask, create an account, and obtain some SepoliaETH.

Install MetaMask

1. Go to <https://metamask.io> and download the extension for your browser (Firefox, Chrome, or Brave).
2. Follow the setup wizard: choose *Create a new wallet* and set a strong password.
3. MetaMask will generate a mnemonic phrase (12 words). Write it down on paper and store it safely.

Important! Never share your mnemonic phrase or private key with anyone. Whoever has it has full control over your funds.

4. Confirm the mnemonic phrase when prompted to complete the setup.

Configure the Sepolia testnet

By default MetaMask shows the Ethereum mainnet. To switch to Sepolia, do the following.

1. Click the network selector at the top of the MetaMask window (it may say *Ethereum Mainnet*).
2. Enable *Show test networks* in *Settings* → *Advanced* if Sepolia is not visible.
3. Select *Sepolia test network*.

Your *address* (derived from your public key) is shown at the top. This is your identity on the blockchain.

Get SepoliaETH from a faucet

1. Go to <https://sepolia-faucet.pk910.de>.
2. Enter your MetaMask address and start mining. Wait until you have accumulated at least 0.2 SepoliaETH before stopping (this may take a few minutes).
3. Check that the funds have arrived by looking at your balance in MetaMask.

3 Send a transaction

Now you will send a simple ETH transfer transaction to a given address and inspect it in a block explorer.

1. In MetaMask, click *Send*.
2. Paste the following recipient address:

0x5CcaD7354Ec90475a67EB720f2f8398C7D6ae444

3. Enter the amount: 0.1 SepoliaETH.
4. Review the transaction details (amount, gas fee) and click *Confirm*.
5. Wait for the transaction to be mined (usually under 30 seconds on Sepolia).
6. Open the block explorer at <https://sepolia.etherscan.io> and search for your address or the transaction hash. Inspect the transaction: sender, recipient, value, gas used, and block number.

4 Deploy a smart contract

The SimpleStorage contract

We will write the *hello world* of smart contracts: a contract that stores a single integer and lets anyone read or update it. The contract should:

- Store a single variable of type `uint256` called `storedValue`.
- Expose a function `setStoredValue(uint256 newValue)` to update the stored value (this is a *state-changing* function and will cost gas).

- Expose a function `getStoredValue()` to read the stored value (this is a **view** function and is free).

The Solidity code for this contract is:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract SimpleStorage {
5
6     uint256 storedValue;
7
8     function getStoredValue() public view returns (uint256) {
9         return storedValue;
10    }
11
12    function setStoredValue(uint256 newValue) public {
13        storedValue = newValue;
14    }
15 }

```

Note. Declaring `storedValue` as `public` is equivalent: the Solidity compiler automatically generates a getter function. The following shorter version is identical in behaviour:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract SimpleStorage {
5
6     uint256 public storedValue;
7
8     function setStoredValue(uint256 newValue) public {
9         storedValue = newValue;
10    }
11 }

```

Using Remix to deploy it

1. Open <https://remix.ethereum.org> in the same browser where MetaMask is installed.
2. In the *File Explorer* panel, create a new file called `SimpleStorage.sol` and paste the first version of the contract above.
3. Go to the *Solidity Compiler* tab (icon that looks like “S”) and click *Compile SimpleStorage.sol*. Make sure there are no errors.
4. Go to the *Deploy & Run Transactions* tab (icon that looks like Ethereum logo with an arrow).
5. In the *Environment* dropdown, select *Injected Provider — MetaMask*. MetaMask will ask you to connect: approve the connection.
6. Verify that the account shown in Remix matches your MetaMask address and that you are on Sepolia.
7. Click *Deploy*. MetaMask will ask you to confirm the transaction. Review the gas fee and click *Confirm*.
8. Wait for the deployment transaction to be mined. The contract will appear under *Deployed Contracts* at the bottom of the panel.
9. Find the deployment transaction on <https://sepolia.etherscan.io>. Note the address of your deployed contract (shown in Remix and on Etherscan under *Contract Address*).

5 Interact with your smart contract

Once deployed, you can call the functions of your contract directly from Remix.

1. In Remix, under *Deployed Contracts*, expand your **SimpleStorage** contract.
2. Call **getStoredValue**: click the blue button. Since this is a **view** function, no transaction is needed and no gas is spent. You should see the value 0 (the default).
3. Call **setStoredValue**: type a number of your choice in the input field next to the orange button (e.g. 42) and click it. MetaMask will ask you to confirm a transaction. Note that this call modifies the blockchain state and costs gas. Confirm it.
4. Wait for the transaction to be mined, then call **getStoredValue** again. Verify that the returned value matches the number you set.
5. Find the interaction transaction on <https://sepolia.etherscan.io>. Notice the *Input Data* field: it encodes the function selector and the argument you passed.

In the following figure we summarize the interaction flow between the three tools we used: Remix, MetaMask, and the Sepolia testnet.

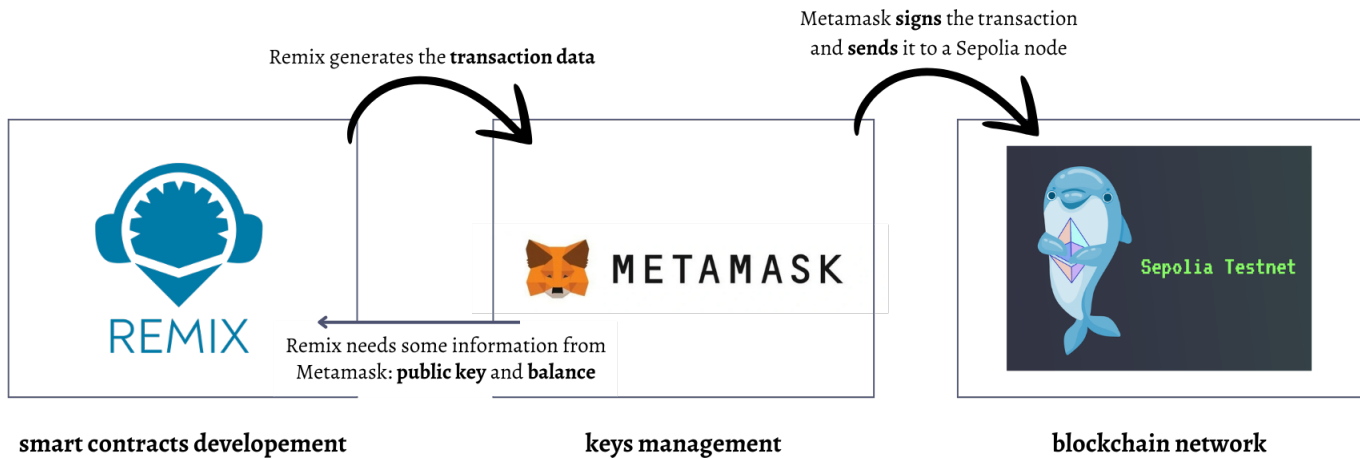


Figure 1: Interaction flow between Remix, MetaMask, and the Sepolia testnet.